

Conducting a Web Application Security Assessment

Internet and Web Application Security 3e - Lab 7

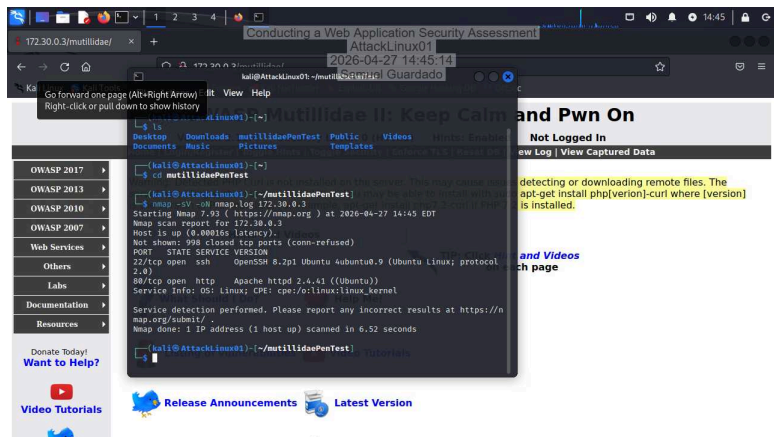
Student: Samuel Guardado
Email: sguar5@brockport.edu
Report Generated: Thursday, July 30, 2026 at 5:19 PM

Time on Task: 2 hours, 11 minutes
Progress: 100%

Hands-On Demonstration

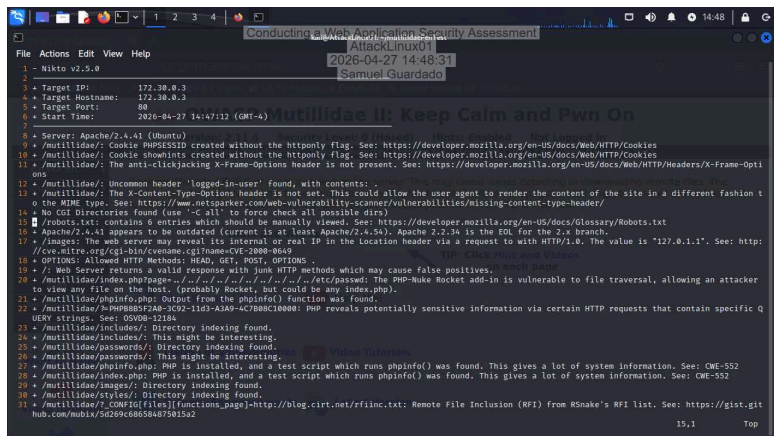
Part 1: Scan the Web Application for Vulnerabilities with Nmap

10. Make a screen capture showing the results of the Nmap scan.



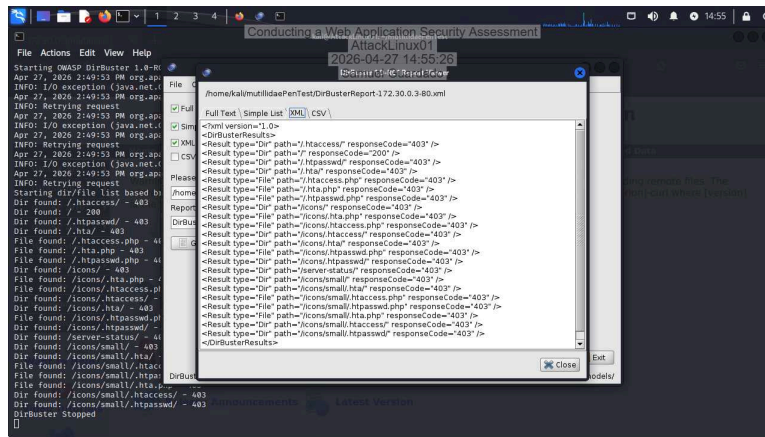
Part 2: Nikto Web Server Assessment

5. Make a screen capture showing the Nikto results log.



Part 3: Directory Discovery with DirBuster

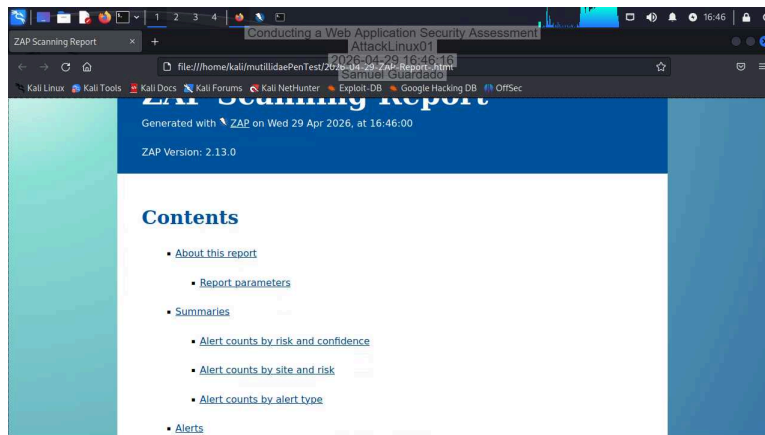
13. Make a screen capture showing the XML version of the report.



Applied Learning

Part 1: Web Application Vulnerability Assessment

17. Make a screen capture showing the report.



Part 2: Evaluate Web Vulnerabilities

1. Document findings for at least five of the vulnerabilities that OWASP ZAP found.

Conducted a system scan for a partial crawl (30%) using OWASP ZAP finding the following vulnerabilities:

Cross Site Scripting(Risk: High) allows attackers to inject malicious JavaScript into web pages viewed by others.

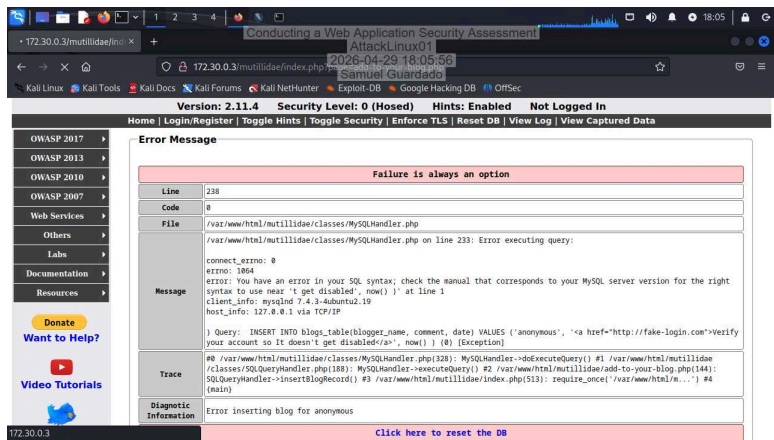
SQL Injection(Risk: High) attackers can manipulate the database queries by injecting malicious SQL code. The OWASP scan showed an alert of a stored XXS payload and a Malicious SQL Query. This can lead to theft of session cookies or user data, hijack of user sessions and altering page behavior. Some recommendations for these two would be enforcing input validation, auto-escape output default, and suing Content Security Policy. Missing Anti-Hijacking header(Risk: Medium) makes the application vulnerable to clickjacking attacks. The scan shows missing X-Frame options header. A click jacking attack could lead to stealing sensitive information, stolen credentials, UI manipulation and loss of user confidence if an account is compromised. Some recommendations would be adding X-Frame Options, and developing a Content-Security-Policy.Next, a vulnerable JS Library(Risk: Medium) the scan alerted an outdated library version. This can produce exploitable client-side vulnerabilities, makes the application easier to exploit, scripts can be manipulated, and can indicate the user there is poor maintenance. Updating to the newest version of the library and regular audits can help with this issue.Absence of Anti-CSRF Tokens(Risk: Medium) the scan indicated missing anti-CSRF tokens in forms or request, which are needed to identify legitimate user requests, producing a vulnerability for XXS request forgery attacks. This can lead to unauthorized actions, changes, transactions without the users consent, finally resulting in the user losing trust on the platform.The Mutillidae web application contains multiple medium to high risk vulnerabilities, which can bring serious threats to the data confidentiality and system integrity, if not addressed, it

could result in a data breach. Some recommendations would include forcing input validation, having more secure code practices, patch management often, and adding security headers.

Challenge and Analysis

Part 1: Identify Phishing Vulnerabilities

Make a screen capture showing that you’ve successfully identified a phishing vulnerability.



Part 2: Exploit the System

Make a screen capture showing that you’ve successfully exploited a phishing vulnerability.

